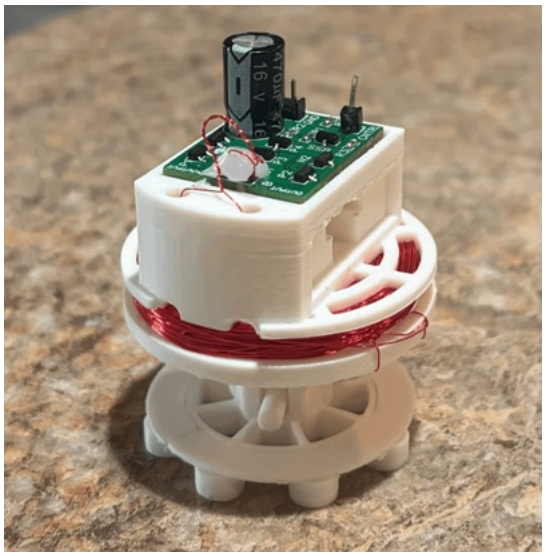
Metal Orchestra Toggle Actuator

ARAM-MOTCA_0_10-0SP*

Quick Start Guide



Product of American Robotics Assisted Manufacturing
www.ARAM-USA.com
View further kit usage information & legal information

Table Of Contents

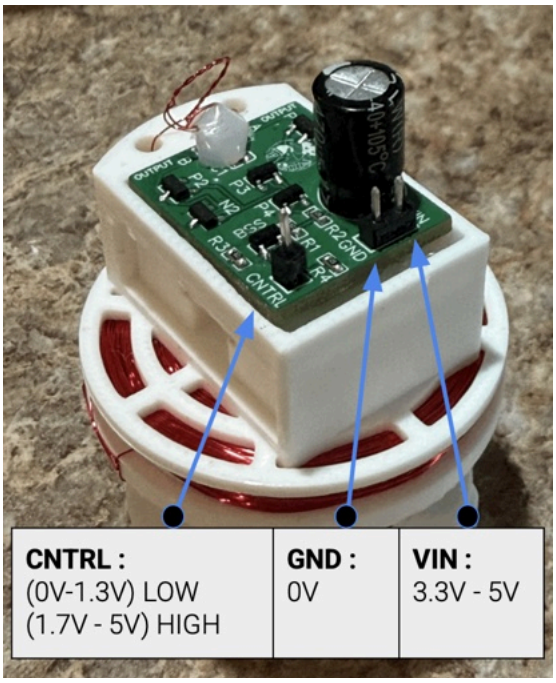
Table Of Contents & Introduction	1
Pinout Diagram	2
Using With A Breadboard: Basic Operation	3,4
Using With A Breadboard: Basic Operation With Buttons	5,6
Using With An ESP32/Arduino: Basic Operation	7,8
Using With An ESP32/Arduino: Actuator Library	9
Using With An ESP32/Arduino: Actuator Library **-** Using The Actuator	10
Using With An ESP32/Arduino: Actuator Library **-** Generating A Beat	11
Further Uses : Instrument Library, Bluetooth MIDI Instrument	12
Conclusion : Thank You, Contact Information, Legal Disclaimer	13

Introduction

Thanks for using ARAM's Metal Orchestra Kit. In here you'll find everything you need to get started. Including simple usage with buttons, breadboards, and microcontrollers, (ESP32/Arduino). Use this actuator with household metal items to create an orchestra. In the companion "Metal Orchestra Quick Start Guide" view details on using these actuators to create a bluetooth MIDI instrument

*Please read the full spec sheet and view legal disclaimers.

Pinout and Basic Usage

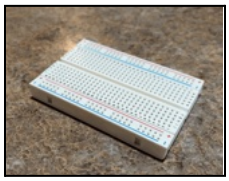


Pin	Voltage	Additional Comments
VIN	3.3V to 5V Continuous 5V+ to 12V Pulsed	Incorrectly plugging VIN to 0V may damage your unit
GND	0V	Incorrectly plugging GND to voltage may damage your unit
CNTRL	(0V - 1.3V) Low (1.7V - 5V) High	(Default) Pulldown to 0V *High Signal* will cause strike *Low Signal* will cause retract

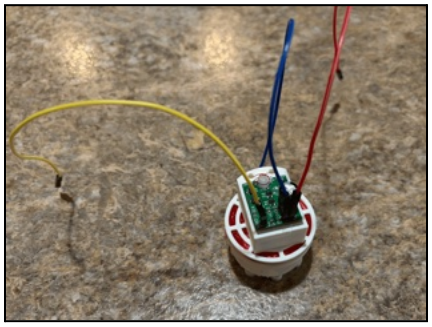
Using With A Breadboard: Basic Operation

Lets confirm the basic operation of your actuator:

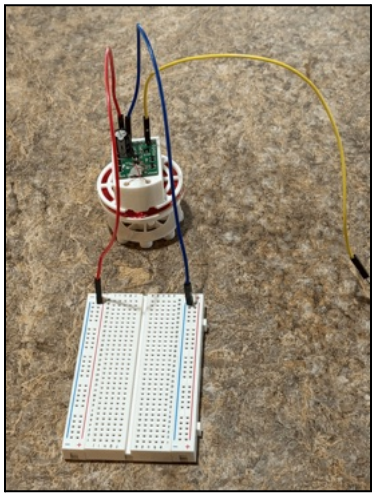
You'll need:

				
1 x **Power Supply** *3.3V - 5V* *capable of 0.5 Amps*	1 x **Solderless Breadboard**	3 x **Male to Male Jumper Wire**	3 x **Male to Female Jumper Wire**	1 x **ARAM-MOTCA Toggle Actuator**

First let's attach jumpers to the actuator:

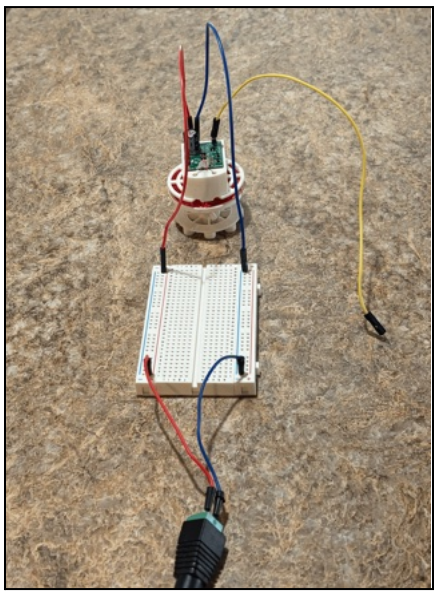


- Plug red/white lead into VIN
- Plug blue/black lead into GND
- Plug the remaining color lead into CNTRL



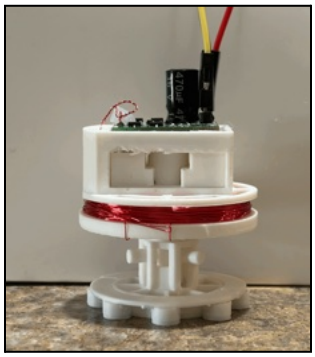
Now plug the actuator into the breadboard

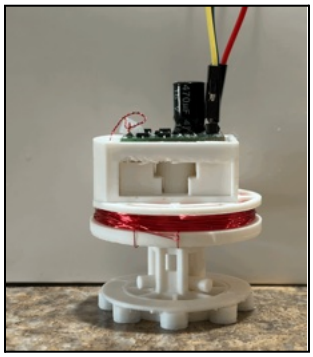
- Plug the red/white lead into the rail labeled (+) as shown in the image.
- Plug the blue/black lead into the rail labeled (-) as shown in the image.



Next we connect the power supply:

- Connect positive (+) from the power supply to the positive (+) rail of the breadboard
- Connect negative (-) from the power supply to the negative rail of the breadboard
- ***Before plugging the power supply in:***
  Double check all connections are correct





Your actuator is ready to use!

The actuator has two states:

Strike
Retract

To make the actuator strike, grab that last wire, the CNTRL lead.

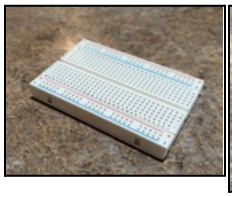
- If you plug this lead into (-) the actuator will **retract**
- If you plug this lead into (+) the actuator will **strike**

The CNTRL pin will automatically go to 0V (ie: the negative rail voltage) if you forget to plug it into anything.
Therefore the default behavior for the actuator is to be in the retract position.

Using With A Breadboard: Basic Operation With Buttons

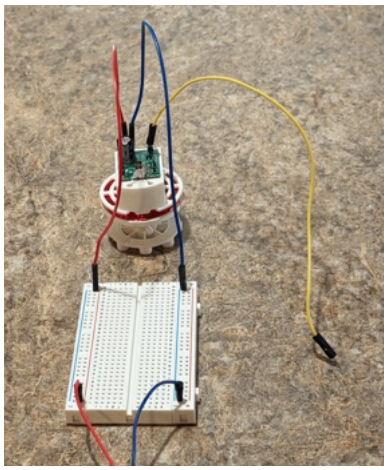
Let's make a basic button control board for your actuator:

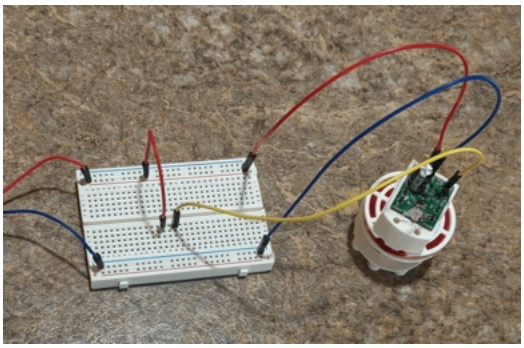
You'll need:

					
1 x **Power Supply** *3.3V - 5V capable of 0.5 Amps*	1 x **Solderless Breadboard**	4 x **Male to Male Jumper Wire**	3 x **Male to Female Jumper Wire**	1 x **ARAM-MOTCA Toggle Actuator**	1 x **Push Button**

First wire VIN and GND to the breadboard, along with the power supply like in the previous page.

- Don't power the supply on yet





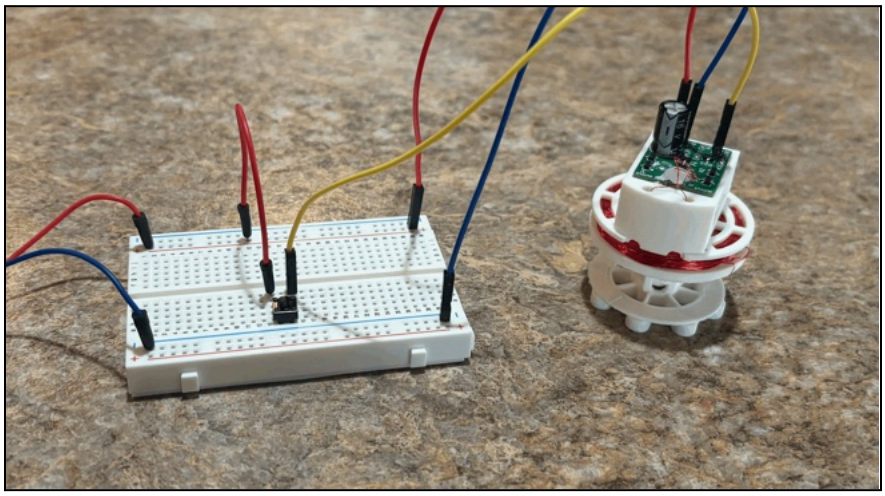
Now build your button circuit

- Using a jumper connect the positive (+) rail to any rail in the middle of the breadboard

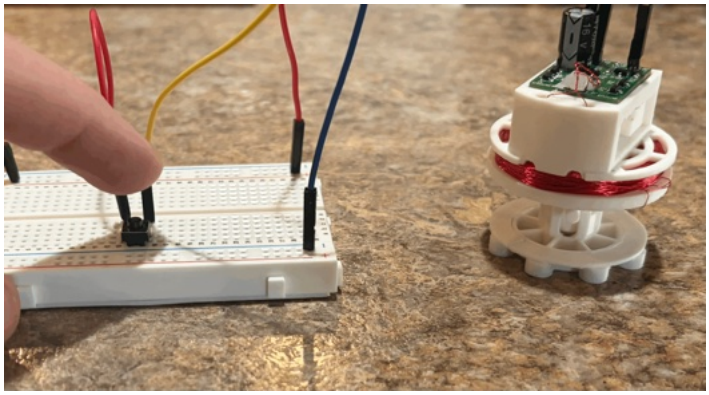
- Connect the CNTRL wire two rails away as shown in the image

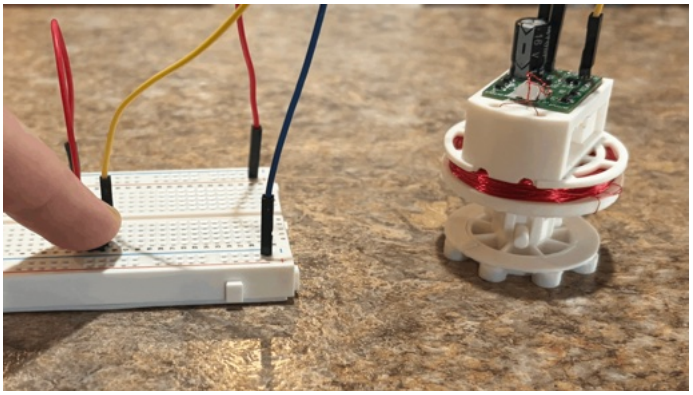
Now place a button on the breadboard bridging the two rails.
This completes the button circuit

Always double check your connections before plugging in your power supply



Simply press the button the strike, and release the button to retract

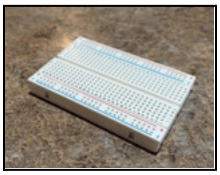




Using With An ESP32/Arduino: Basic Operation

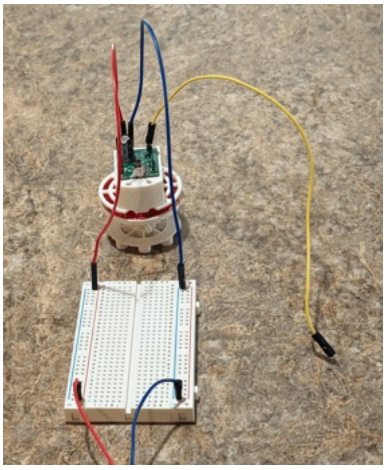
This is the basic wiring for all of the ESP32/Arduino Projects shown here

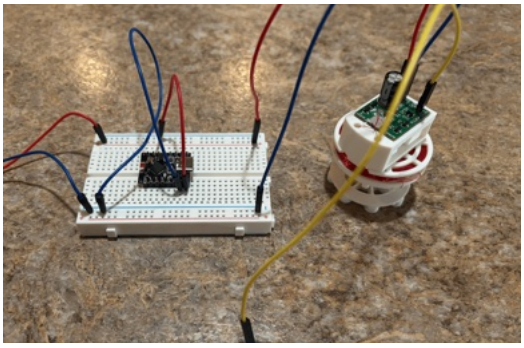
You'll need:

					
1 x **Power Supply** *3.3V - 5V capable of 0.5 Amps*	1 x **Solderless Breadboard**	4 x **Male to Male Jumper Wire**	3 x **Male to Female Jumper Wire**	1 x **ARAM-MOTCA Toggle Actuator**	1 x **Push Button**

First wire VIN and GND to the breadboard, along with the power supply like in "Using With A Breadboard: Basic Operation"

- Don't power the supply on yet



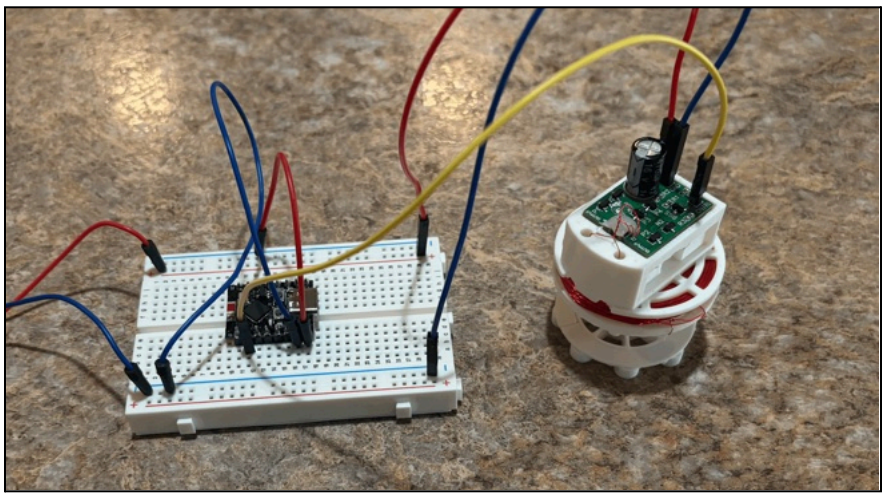


Now place the ESP32/Arduino In the middle of the breadboard

- Using a jumper connect the positive rail of the breadboard (+), to either VIN (if using 3.3V power supply) or to the 5V pin (If using a 5V power supply).
- Using a jumper connect the negative rail of the breadboard (-), to the GND pin

Now simply connect the CNTRL pin of the actuator to whichever pin of your ESP32/Arduino which you want to control the actuator

Always double check your connections before plugging in your power supply



Now the ESP32/Arduino is ready to use the actuator.

You may strike the actuator by calling:
pinWrite(cntrl_pin, HIGH)

You may retract the actuator by calling:
pinWrite(cntrl_pin, LOW)

However the next section covers the implementation of the ARAM_TOGGLE_ACTUATOR library which streamlines the use of these, without needing to call pinWrite directly.

Using With An ESP32/Arduino: Actuator Library

For ease of use ARAM has released a library compatible with our toggle actuators.

To use the library simply type “ARAM_Toggle_Actuator” into the Arduino IDE library manager, and select the entry by that name.

Getting Started

To import the library into your code use:

```
#include “ARAM_TOGGLE_ACTUATOR.h”
```

To declare a new actuator use:

```
ARAM_TOGGLE_ACTUATOR myActuator;
```

To initialize the actuator use

```
myActuator = ARAM_TOGGLE_ACTUATOR( Cntrl_pin );
```

Or if you intend to use it with the instrument library you must also declare a frequency.

```
myActuator = ARAM_TOGGLE_ACTUATOR( Cntrl_pin , frequency_in_hertz );
```

(To precisely determine the frequency simply use an audio analyzer app to determine which frequency is loudest when the actuator strikes its target)

Basic Operation

Use the following the make the actuator strike/retract

```
myActuator.strike();
myActuator.retract();
```

Advanced Operation : Update

To use beat, vibrate, or schedule functionality, you must call the following function inside the loop:

myActuator.update();

Advanced Operation : Beats

You can cause the actuator to generate a beat using

myActuator.generateBeat(strike_time, beat_timing);

Here strike_time is the amount of time the actuator should be striking for, in milliseconds
Here beat_timing is the amount of time between beats, in milliseconds
generateBeat(100,1000) -> generates a strike for 100ms every 1 second

Stop the beat using

myActuator.stopBeat();

Advanced Operation : Vibration

You can cause the actuator to vibrate a specific frequency using

myActuator.vibrateAt(frequency);

You can stop the vibration using

myActuator.endMovement();

Advanced Operation : Actuator Modes

Putting the actuator into standard actuator mode lets you use all functionality, also the actuator always ends in the retract positon

myActuator.setActuatorMode(ARAM_TOGGLE_ACTUATOR_Actuator);

Putting the actuator into frequency generator mode restrict you to vibrate functionality, also the actuator always ends in the strike positon

myActuator.setActuatorMode(ARAM_TOGGLE_ACTUATOR_Frequency_Generator);

Using With An ESP32/Arduino: Actuator Library - Using the Actuator

As described in the earlier section, using the actuator with the ARAM_Toggle_Actuator library is incredibly easy. This example code shows declaring an actuator, and then causing it to strike and retract in a repeating motion.

```
//import ARAM Toggle Actuator Library
#include "ARAM_Toggle_Actuator.h";

//define what pin we're gonna use
#define CNTRL_PIN_A 1     //change me to your desired cntrl pin A

//declare  toggle actuator
ARAM_TOGGLE_ACTUATOR a1;

void setup() {

 //explicitly set pin to output mode <- may not be necessary
 gpio_reset_pin(GPIO_NUM_1);

 //initialize toggle actuator
 a1  = ARAM_TOGGLE_ACTUATOR(CNTRL_PIN_A);
}

void loop() {

   //make the actuator strike
   a1.strike();

   //wait 75 ms
   delay(75);

   //retract actuator
   a1.retract();

   //wait 75 ms
   delay(75);
}
```

Using With An ESP32/Arduino: Actuator Library - Generating A Beat

This example code shows declaring an actuator, and then causing it to generate a beat. Notice how update() is called in the loop function.

```
//import ARAM Toggle Actuator Library
#include "ARAM_Toggle_Actuator.h";

//define what pin we're gonna use
#define CNTRL_PIN_A 1     //change me to your desired cntrl pin A

//declare toggle actuator
ARAM_TOGGLE_ACTUATOR a1;

void setup() {
 //explicitly set pin to output mode <- may not be necessary
 gpio_reset_pin(GPIO_NUM_1);

 //initialize toggle actuator
 a1  = ARAM_TOGGLE_ACTUATOR(CNTRL_PIN_A);

 //have the actuator generate a beat
 a1.generateBeat(100, 500);
}

void loop() {
 //in the loop we must call update on each actuator in order to use the beat functionality
 a1.update();
}
```

Further Uses: Instrument Library, and Bluetooth MIDI Instrument

Bluetooth MIDI Instrument :

Once you've familiarized yourself with the actuator, you can take it to the next level by creating a Bluetooth MIDI instrument. Follow the guide in the companion guide, "Metal Orchestra Quick Start Guide", to see how this is done.

Instrument Library:

This library is general purpose to turn an array of actuators into a playable instrument. This is what we use for our Bluetooth MIDI Instrument, but it can be used for any other project too. View the companion guide, "Metal Orchestra Quick Start Guide", to see how this is used.

Additional Resources:

https://github.com/ARAM-USA-Support/ARAM-Metal-Orchestra
(Metal Orchestra Github Documentation Mirror)
https://github.com/ARAM-USA-Support/ARAM-MOTCA_0_10-0S
(MOTCA-1 Toggle Actuator Github Documentation Mirror)
*All documentation is also available at www.ARAM-USA.com

For all projects, guides, documents, and questions please:
View Our Website : www.ARAM-USA.com
Contact Us At: support@ARAM-USA.com

DISCLAIMER. – ALL PRODUCTS SOLD BY ARAM (AMERICAN ROBOTICS ASSISTED MANUFACTURING) ARE PROVIDED “AS IS” TO THE FULLEST EXTENT PERMITTED BY APPLICABLE LAW, ARAM EXPRESSLY DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. ARAM MAKES NO WARRANTY THAT THE PRODUCT WILL MEET YOUR SPECIFIC REQUIREMENTS OR THAT IT WILL OPERATE WITHOUT INTERRUPTION OR ERROR. All units are tested prior to sale; however, ARAM does not warrant that every unit will arrive in working condition. Electrical properties and specifications provided in this document are for reference purposes only. It is the sole responsibility of the user to verify that the specifications provided herein match the electrical characteristics of the unit received. Use of this product outside of stated operating parameters is at the user’s own risk.

Sole Remedy – Replacement Only. ARAM takes no responsibility for any damages caused by the use of our product, whether inside or outside standard operating conditions. In the event of a defect or failure, ARAM’s sole obligation, and your exclusive remedy, shall be the replacement of the defective unit within thirty (30) days of the original purchase date. To qualify for replacement, the buyer must contact ARAM within this period and is responsible for all shipping and handling costs associated with the return and replacement.

Limitation of Liability. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL ARAM BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, OR PUNITIVE DAMAGES OF ANY KIND ARISING OUT OF OR RELATED TO THE USE OR INABILITY TO USE THIS PRODUCT. ARAM’S TOTAL CUMULATIVE LIABILITY TO YOU FOR ANY AND ALL CLAIMS ARISING OUT OF OR RELATED TO THIS PRODUCT SHALL NOT EXCEED THE ORIGINAL PURCHASE PRICE PAID FOR THE SPECIFIC UNIT GIVING RISE TO THE CLAIM. THIS LIMITATION OF LIABILITY SHALL APPLY EVEN IF THE SOLE AND EXCLUSIVE REMEDY PROVIDED HEREIN IS FOUND TO HAVE FAILED OF ITS ESSENTIAL PURPOSE, AND SHALL APPLY WHETHER THE CLAIM IS BASED IN CONTRACT, TORT, STRICT LIABILITY, OR ANY OTHER LEGAL THEORY.

Important Note. Some states, including New York, may have consumer protection laws that limit the enforceability of certain warranty disclaimers. Nothing in this disclaimer is intended to exclude or restrict any rights you may have under applicable state law that cannot be waived by contract.

Thank you for purchasing an ARAM product

We would like to thank you for purchasing an ARAM product. We highly value customers like you. If you have any feedback, suggestions, or questions, please reach out to our contact email. *If your unit arrives with any issues please contact us within 30 days for a replacement.* We would like to thank you for supporting ARAM, American Robotics Assisted Manufacturing, a 100% American based work force and company.

Sincerely:

Jack Serlin,
Founder of American Robotics Assisted Manufacturing

Contact Email:

Support@Aram-usa.com